Reflections on SOA and Microservices

Zhongxiang Xiao
School of Electronic Engineering
Xi'an Shiyou University
Xi'an, China
xzx@xsyu.edu.cn

Inji Wijegunaratne
Enterprise Architect
Infosys Australia
Melbourne, Australia
inji_wijegunaratne@infosys.com

Xinjian Qiang
School of Computer Science
Xi'an Shiyou University
Xi'an, China
xasyu@126.com

***Abstract*—Today's Enterprises are facing many challenges in the service oriented, customer experience centric and customer demand driven global environment where ICT is becoming the leading enabler and partner of the modern enterprise. In the last decade, many enterprises have invested heavily in SOA-aligned IT transformations, but not harvested what SOA promised to provide. Now the API and Microservice paradigm has emerged as the "next big thing" for delivering IT outcomes to support the modern enterprise, with many technology vendors and service jumping on the bandwagon. This paper undertakes a critical investigation of the key concepts around SOA, API and Microservices, identifying similarities and differences between them and dispelling the confusion and hype around them. Based on our discussion and analysis, this paper presents a set of recommendations and best practices on the effective use and management of enterprise software components, drawing upon the best of SOA, API and Microservice concepts and practice.**

Keywords- service oriented architecture; microservice; API; enterprise architecture; architecture design pattern

I. INTRODUCTION

Modern integration may arguably be traced back to the paper in 1984 by Birrell and Nelson [1] on the remote procedure call. Since then there have been many environments, products, styles - DCE (Distributed Computing environment), CORBA (Common Object Broker Architecture), two tier client/server, three tier client/server, n-tier client/server, COM, DCOM, messaging and message-oriented middleware, service-oriented architecture, API-based integration, Micro services.

Progress has been linear in one sense – integration has consistently become easier with the passage of time – what was horrendously difficult with DCE for instance, is now relatively simple, with underlying platforms presenting simple interfaces, mechanisms such as XML, JSON, providing simple ways of describing the interface, and the likes of HTTP presenting simple transport protocols.

However, the evolution of integration styles and practice has been cyclic, with many concepts falling out of favour but reappearing with the passage of time under different guises. We see some elements of the arguments for two-tier client/server in today's push for micro services, elements of best practice under CORBA and COM reappearing, today's concept of "smart endpoints and dumb pipes" reminiscent of the early days of EAI, etc. We also see various gurus – with a vested interest in keeping alive an integration debate – making pronouncements from time to time.

This unfortunate trend is evident in strands of the current discourse on SOA and micro services that discuss the "better" style, with ideology, fashion and vested interest appearing to trump common sense and reason. For example Gartner's hype cycle for application architecture shows that micro services are currently at the peak of the hype cycle [2]. At the peak, "inflated expectations", the tendency is to treat the technology trend as a cure-all for all IT problems, leading to the "trough of disillusionment". Reality then kicks in, maturity around the concept grows, and gradually the realisation dawns on what the technology can and cannot achieve. Observe, for example, the transition of the older integration style, SOA, through the hype cycle from 2002-2010.

In this reflections paper, we intend to take a reasoned and dispassionate approach to the question. We will reflect upon best practice for architecting, designing, and developing application program interfaces from the perspective of desired outcome – what is best to achieve particular types business outcome.

II. SOA

Service-Oriented Architecture (SOA) is obviously the older of the two styles, making its presence felt in the early 2000s and reaching a level of maturity by the end of the first decade of the millennium. See the Gartner hype-cycle below.

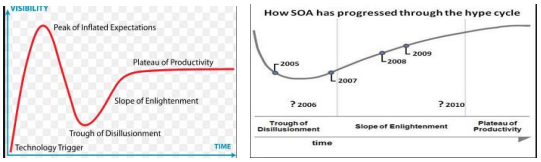

Figure 1. Gartner hype-cycle and SOA progress [2].

978-1-5090-3790-2/16 $31.00 © 2016 IEEE
DOI 10.1109/ES.2016.14

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

SOA approach and products came about through a need to create a component based distributed architecture that promotes business agility – ie be able quickly to configure a solution to address a particular business need and to provision changes to the solution as the business needs evolve rapidly.

Definitions abounded. Some of the early definitions are:

- IBM: An application framework that takes everyday business applications and breaks them down into individual business functions and processes, called services.
- Wikipedia: An architectural pattern in computer software design in which application components provide services to other components via a communications protocol, typically over a network. The principles of service-orientation are independent of any vendor, product or technology.

Componentisation (into services), loose coupling, high performance/ throughout, service lifecycle management (ie managing the services – taking central custody, promoting reuse, managing their lifecycle events through creation to eventual deprecation and retirement) were essential features of SOA; but none was more important than the design of these services – the repeated mantra was that these were business services, and a key prerequisite to SOA success was their proper architecture and design.

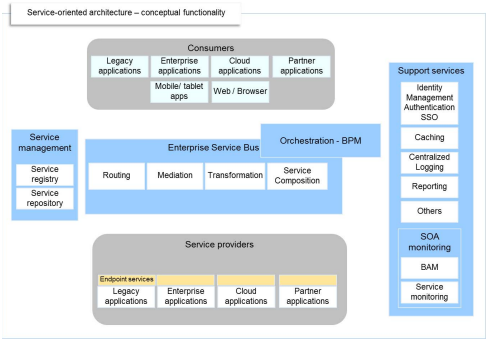

Figure 2. Components of a Service-oriented architecture.

The core components in a SOA architecture are:

- SOA services: A service is a piece of self-contained business functionality: that is, a service in SOA is designed to encapsulate some functionality that is meaningful to the business. The functionality might be simple (as in retrieving a customer address) or complex (eg encapsulating the business process for fulfilling a customer order).
- Enterprise service bus (ESB): The infrastructure that enables high interoperability: it mediates between service providers and consumers, provides value added services and offers a platform of high throughput, reliability, scalability (of course dependent on the infrastructure) between providers and consumers.
- Service management and monitoring: Service management via a service registry/ repository provides service discovery, lifecycle management and governance functions; the enabling functions essential to efficiently manage significant numbers of services. Facets of runtime performance are monitored and reported on through the monitoring capabilities.

These components and functionality are underpinned by the key concept of loose coupling. This is a fundamental principle in SOA – designing to minimise dependencies between SOA components to the greatest extent possible. This finds expression in uncoupling service providers and consumers through service proxies and the ESB; in abstracting service descriptions, and most of all, in the design of SOA services.

Use of a service model is a key element in service design. Unfortunately a commonly agreed standard service model does not exist, with most SOA product vendors and system integrators promoting their versions of service models that generally have a common core typology with some variation at the "edges". A service model the authors have found useful is described below.

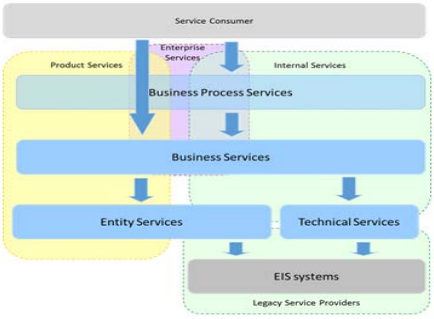

Figure 3. A sample service model.

Service Category	Description
Enterprise Services	Those core services which are considered to be re-usable across the enterprise. Enterprise services should not be at the technical services layer and must be aligned with the enterprise's common information model (CIM). Enterprise services may be purpose-built at the business process or business service layers, or selected services from the product and internal services categories may be chosen and re-purposed.
Internal Services	These include internal channel or project services and services at the technical layer. These services can be owned by a business unit, but in general are not considered for re-use across the enterprise, and are not required to be CIM compliant
Product Services	These are services provided by COTS products. Product vendors are able to deliver OOTB services across all the layers dependent on the product's alignment with the organisation's business.

Layer	Description
Business Process services	The Business Process Layer defines the sequencing and events of the business process. Process Services are stateful services representing the whole or a fragment of one of the organisation's business processes
Business Services	Business Services represent stateless services which directly implement business service capability and

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

	provide business value. This layer is where there is also a focus on the use of a common messaging model or common information model When invoked from a Business Process Service, a Business Service represents a specific task within the business process.
Entity Services	Entity Services are built to handle persistent data and manage all associated functions such as storing and retrieving, locking and transaction management. Such a service will manage a single data entity only and strictly encapsulates that data entity.
Technical Services	Technical services provide support to business functions, rather than being a specific business service themselves. Technical services can be linked to form business services composites, using process orchestration or service assembly A sub-layer within Technical services provides the support services which are those services providing the platform capabilities such as logging and exception handling

III. MICROSERVICES

A microservice is described as a (small) application in its own right, able to evolve independently and choose its own architecture, technology, and platform, and can be managed, deployed and scaled independently with its own release lifecycle and development methodology [3]. Accordingly, a microservice based architecture is defined as an architecture pattern for distributed applications where the application is comprised of a number of smaller "independent" components that are small application in themselves [3]. For example, envisage an online ordering application. This application performs multiple functions such as providing address validation, a product catalogue, customer credit check, etc. With the microservice architecture pattern, individual applications are created for address validation, customer credit check and online ordering; these application are then grouped together to create the ordering application (which now is a composite application). This approach to application development challenges the traditional "monolithic" application and services.

Hence a microservice is built around the scope of "business-bounded context, and its concerns will apply to that context. As in the example above, a microservice can then be mapped to enterprise systems such as customer relationship management, payroll, etc or to smaller functions within these systems.

A microservice must be "independently deployable" with each microservice having its own deployment architecture; and not share its resources like containers, caches, datastores with other enterprise software components. For instance, where the microservice stores and retrieves information from a database, the database schema should be part of the microservice. Similarly, scaling of one microservice is independent of other microservices; and increasing the capacity of a microservice, by moving its hosted target or by increasing the number of instances, should not have any impact on its consumers [5].

Applications expose interfaces that can be used by other applications to interact with are defined as "application programming interfaces", and a microservice provide its own interfaces for communications. Microservice APIs are built using internet communication protocols such as HTTP and adhere to open standards such as REST and use data exchange technologies like JSON [6].

In an enterprise, several teams would require access to create or enhance common functions like customer account. To foster a clear understanding of a microservice, so that they may be reused effectively, specifications must be defined, maintained in an enterprise wide shared repository of all the microservices specifications. Additional information like status, ownership, can also be included. Interfaces provided by microservices should be discoverable: consumer of the interfaces must be able to look up, find the interfaces without having explicitly knowledge of the underlying technology implementation or location.

Independence is a key characteristic of microservices architecture. To enable this characteristic a microservice should have stewardship of the data within its context; other microservice would allowed to read its data; however writes would be performed through the service that possesses stewardship of the data. For instance a customer account microservice is the data steward for customer accounts, with other microservices being able access read-only versions of customer account data.

A microservice normally comprises of three layers as a typical 3-tiered application, consisting of an interface layer, a business logic layer and a data persistence layer, but within a much smaller bounded context. This sets a broad scope of the technical capabilities that a microservice could possess. However not every microservice provides all capabilities; this would vary on how the function provided is meant to be consumed. For example, a microservice used primarily by providers of APIs would have communications interface layer, business logic and data persistence layers but not necessarily a user interfaces.

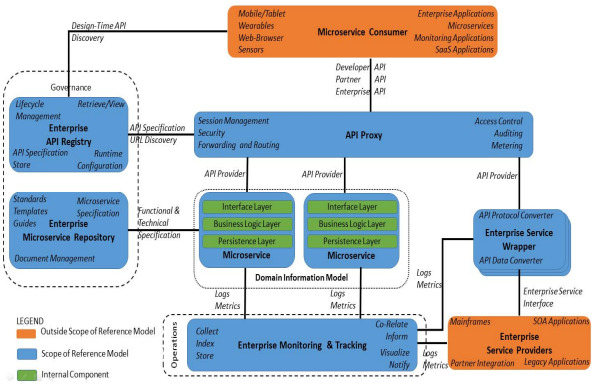

Figure 4. Reference microservice architecture [4].

A reference architecture model for microservices is presented above. This model consists of key architectural components and building blocks for implementing and managing microservices in the context of enterprise architecture within an organization. These components are as follows.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

A. *Microservice Domain*

When this microservice based architecture model is applied at an enterprise level within an organization or company, the resultant architecture of this application decomposition would consist of fine-grained building blocks, creating a proliferation of smaller applications or microservices. A "microservice domain" normally is a logical business entity that contains a collection of related microservices. Its purpose is to assist in managing the microservices of that domain, especially dealing with the potential proliferation of microservices and its consequences such as multiple data models, platform and technology proliferation, cross-dependencies to other microservices, etc. The business functional domain serves as serve as the boundary of the business "container"; eg microservices supporting billing functions will be managed and governed by the business unit associated with billing processes and activities. Techniques such as a "domain information model" may be employed to foster consistency and interoperability of microservices and development standards within a domain.

B. *Enterprise API Registry and Repository*

The "discovery" requirements of the microservices are met through the use of an API registry service. Its purpose is to make visible the interfaces exposed by the microservice to consumers of the services both within and outside the enterprise. Its information content is published in a standard format, and access controlled. It possesses search and retrieval capabilities enabling design time look up of API specifications. It provide a central location for governance of the enterprise API portfolio.

C. *API Proxy*

The proxy or surrogate, "a placeholder for another object to control access", decouples the "public" access point from the origin or endpoint, enabling the protection of the underlying resources, providing a known location for remote resources, enabling throttling, etc. Typically, organizations provide APIs to a different consumers within and outside the enterprise; for example Enterprise APIs to internal consumers, partner APIs to business partners, and public APIs to the general public. The proxy function enables the provision of different SLAs, security requirements, access levels, etc. The API's provided the proxy function can be grouped into three categories.

D. *Enterprise Service Wrapper*

This component enables access to older services (eg SOA services) in the enterprise providing translation between the relevant protocols and data formats. A key component in quickly unlocking the core business capability for composition thru microservices.

E. *Enterprise Monitoring and Tracking Manager*

The enterprise monitoring and tracking manager requires that all emitters (microservices and other applications) publish standardized events (operational and log events, metrics) which it would gather and store. It would serve as a central collection of all events, on which various analytics operations can be performed.

IV. COMPARATIVE ANALYSIS

As we have pointed out, SOA and microservices do not sit at the same level in a maturity curve. The former has reached its trough in the hypecycle and is now progressing upwards along the slope of enlightenment, whereas the latter is very much the new kid on the block climbing the peak of inflated expectations. In the preceding Sections we described the intent, principles, rationale, and functionality associated with the architectural styles of SOA and microservices without additional commentary. Here we take a critical look at the two styles.

The first point of note is the similarity between the two approaches. Both have the notion of a "service" at their core, a piece of software that follows a request response protocol. The components that support the two architectural (refer to Figures 2 and 4) styles, as shown below, are remarkably similar.

Function	**Component (SOA)**	**Component (Microservice)**
Point of central management; registered services/APIs searchable and discoverable.	Service registry, service repository	Enterprise API registry; enterprise microservice repository
Where services/ APIs are proxied or made available for consumption	Enterprise service bus	API management layer (API proxy)
Endpoint of the service or microservice	Service provider	Microservice provider
Other applications that require the functionality provisioned by the API or service.	Service consumer	Microservice consumer
Runtime service / API monitoring and management	BAM, service monitoring	Enterprise monitoring and tracking

They diverge on the nature of the service and the supporting protocols . The characteristics of the two service styles are described below. The areas of difference are shown against a reddish background, while a green background represents areas of similarity.

SOA Service	**Micorservice**
Services may be fine or coarse-grained	Fine grained, "small", services
Loosely coupled, cohesive	Independent, un-coupled service running in its own container; independently scalable
Centralised governance, governance seen as adding strategic value	Decentralised governance
Very rapid development; quick replaceability.	Considered design and development; designed for durability, well-defined change control
Protocols: HTTP, JMS, other vendor specific messaging; SOAP, XML	Protocols: HTTP; REST, JSON

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

Organised around business capabilities; business-significant boundary	Organised around business capabilities; business-significant boundary
Componentized service as the core offering: Stateless, reusable, abstracted	Componentized service as the core offering: Stateless, reusable, abstracted
Independent of implementation technology, though there are certain prerequisite functions in the supporting framework (See Table above)	Independent of implementation technology, though there are certain prerequisite functions in the supporting framework (See Table above)

Notice that here too, there are several similarities between services of the SOA and microservice styles. The main areas of divergence between the two service styles are the "size", autonomy, and the development cycle associated with the service. Let us investigate them further.

Autonomy: The ideal of the microservice architecture defines each service as an independent unit built on the principle of separation of responsibility, with each service being independent (ie not sharing code or infrastructure), independently scalable, and amenable to rapid deployment and change.

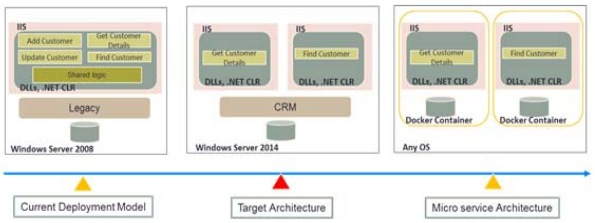

Figure 5. Service deployment.

This model however, is largely infeasible when examined in the context of most organisations today. The leftmost picture shows a typical legacy situation; the middle depicts a typical target architecture with customer functionality migrated to a new CRM package; and at right is the microservice ideal. In this example:

- The legacy deployment model constrains related services to be deployed together, since they share/reuse the same code. Scalability here is achieved by deploying multiple instance of the set of service on new sets of servers.
- The CRM package may enable services to be developed and deployed independently. However, it is extremely unlikely that the package will allow service independence to flow down through to the database level: The services will share the same database (and the schema belonging to the particular module of the CRM package), introducing coupling at that level.

There is an absolutely crucial point here: This is the nature of these products and environments. Some legacy environments will possess these constraints, others may exhibit more. Modern day business system packages typically build modular, configurable, functions on the foundation of a common database schema representing the information characteristics of the business domain. This architecture pattern is employed for good business and commercial reasons: systems organised around sets of cohesive business functions built around a common schema are able to provision business functionality quickly and efficiently. Almost all business system packages are architected this way.

In these environments, a wish to develop independent and autonomous microservices in their own containers down to the database level, is just that – wishful thinking. Blowing apart the typical business system into completely independent microservices is a very costly exercise. To what benefit? Existing systems in your typical enterprise may have any number of shortcomings, but the key objectives of systems replacement projects or larger business transformation programs is to deliver the strategic business objectives in the most cost effective manner, and Business and IT leaders would look first to packaged systems that satisfy the business need. The argument that a vast number of completely autonomous, independently scalable microservices can be constructed to replace say an existing CRM, or any other line of business or back office system would simply not hold water: the ensuing overall complexity will overwhelm any advantage of agility or scalability at individual microservice level.

The second point of difference, fineness of the grain, is also problematic. Sure, fine grained services are possible; in the SOA model, services employed to perform simple CRUD operations on the database (entity service) are fine grained. However, there needs to be business logic between the basic database operations and the functionality exposed to the user of the system. This logic underpins the business operations that front and back office staff perform and the business services extended to the customer. And these – complex relationships between data elements, business rules – reflect the complexity of the business. For example, take a Product Disclosure Statement that typically accompanies your insurance policy; all the rules therein must be available to be applied against your policy when circumstances necessitate it (ie when a claim is made). The granularity of services that provision your business logic ideally needs to correspond to the complexity of the particular piece of business logic that needs to be executed: you may compose a "large", coarse grained service which in turn calls several finer grain services, to answer the question "what physiotherapy benefit am I entitled to for the rest of the year, given my Health Insurance policy and what I have already claimed for Physiotherapy thus far". Accordingly in a services world, the more complex the business the larger the number of coarse-grain services you would expect to see: the granularity of services in your service mix will correspond closely to the complexity of your business. Of course poor design can skew the mix either way – a preponderance of finer or coarser grain services than ideal, but poor service design needs better service design – not a new paradigm. Therefore the implicit expectation in the microservice approach that fine-grained services will suffice to provision most – if not all – needs of the business is badly misplaced.

However, the microservice movement arose to counter certain perceived shortcomings of existing IT; especially that the pace of change associated with traditional IT is not able

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

to keep pace with the rapid demands of the new digital world. In the authors' view this is a legitimate concern, one that must be addressed. The way to address this, however, is not to leave reason behind but to think through the underlying problem and solution options clearly and dispassionately.

V. BEST PRACTICE: A SYMBIOSIS BETWEEN APPROACHES

What then is the best approach to architect, design, develop and govern application program interfaces – "best" here taken in the sense of what is best to achieve the business outcomes of the enterprise?

Ironically, one reason for the genesis of the SOA movement was the perceived lack of agility in the then world of corporate IT caused by monolithic systems, complex environments and integration, poorly understood and supported legacy environments, etc. Unfortunately the same complexity exists today, especially in established businesses – who invariably carry a legacy of systems of varying vintage and design excellence. The demands on agility and speed of response however, has risen almost by an order of magnitude – mostly through the emergence of the Digital world. And this is a very real challenge, especially for established businesses who see upstarts innovating and introducing disruptive business models that take away their customers and profitability. Whereas the asking turnaround 15 years ago was months, today it has become days.

A. Bi-modal or two speed IT

As we described above, the ability to innovate, test and release products to market rapidly, quickly adapt refine and re-release them, is the much sought after ideal for today's digital enterprise. Several technology companies and start-ups have profited from this outlook, bringing disruptive products and services rapidly to market. Today's typical enterprise however, possess core systems – systems that support their core operations – that simply do not possess the characteristics necessary to support this need for quick turnaround and agility – their development and release cycles are in the order of magnitude of months, not hours and days demanded by the digital world.

In the main, today's microservices movement has arisen out of this need – that is, the need to develop, deploy, change services (or API's) rapidly to support today's digital business imperatives. Let us however explore the essential requirement a little further.

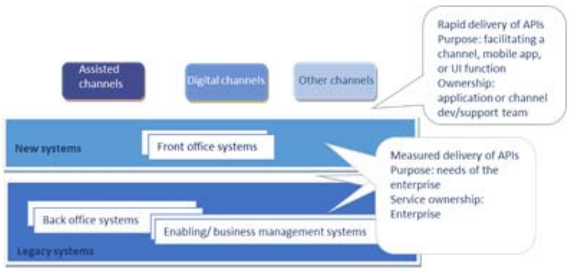

Figure 6. Two-speed IT

As the figure shows, for the reasons we have discussed, today's customer facing channels need rapid delivery of APIs; at the same time the main core systems of the enterprise have complex business rules, complex relationships with other systems, and exhibit a variety of additional features that inhibit rapid development – monolithic and not modular, paucity of documentation, and maintenance and support skills, built at a different time to different design principles relevant to that day and age, etc. Building your small, microservices to replace the existing mass of corporate functionality is not a feasible option. You possibly could, at a massive cost and very long timeframe – by which time you would likely have been disrupted out of your business. Even if, by some miracle, you were to do so, you may rid yourself of the avoidable complexity in the environment caused by legacy constraints, short-sighted design decisions and accumulation of technical debt. However, you will introduce additional complexity arising from the need to manage an estate comprising a very large number of "small" services that evolves very rapidly.

We come back to the point made earlier: a level of complexity is inherent in your business and this is determined by the business environment – factors such as the nature of the business, the competitive and regulatory environment, the size and scale of your business operation, etc. This complexity will translate into the systems that support the business, ie into complexity of business rules, data entities, and relationships between data entities, and be reflected in your collection of APIs or services. The best design approach therefore is to recognise that you cannot wish away this complexity, but need to create a framework that introduces the least amount of additional, avoidable, complexity.

What then is the best approach for going forward, given that we need to support two opposing requirements? One requirement is for rapid delivery and turnaround of services; the other for the more the measured design and delivery needed to navigate our way through business complexity.

The crucial point therefore, is that these two worlds, one for rapid delivery and the other for measured and considered design and delivery need to co-exist. This is where two-speed or bi-modal IT comes into play. According to Gartner [6] "Mode 1 is traditional, emphasizing scalability, efficiency, safety and accuracy. Mode 2 is non-sequential, emphasizing agility and speed." A two-speed IT architecture can be pressed into service to enable companies develop their customer-facing capabilities at high speed while decoupling legacy systems, for which a much more deliberate and considered approach and commensurately sedate release cycles are essential, enabling them to proceed at their own pace.

These two worlds must be loosely coupled, but obviously not entirely separate. We posit that the way the two worlds are joined is through appropriate integration. The integration between the two worlds must be reflected via an architectural and design construct that shows how one spans them in a manner that enables loose coupling while preserving integrity and cohesion. We propose that properly tailored service model that accommodates both styles of services –

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

services that are constructed in a considered, measured manner to satisfy the needs of your core systems, as well as the rapidly deliverable digital APIs – fits this requirement.

B. *A Service model for the two speed architecture*

As we described earlier in this paper, a service model is an organising construct: "Services" or "APIs" are described in the model, which is a taxonomy of service types and relationships between them. We present below, a service model – having its roots in the SOA service model presented earlier in this paper, but modified to accommodate the digital needs of today.

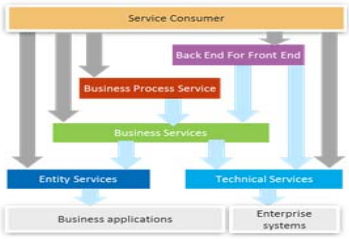

Figure 7. A revised service model

Layer	Description
Entity Services	Entity Services are built to handle persistent data and manage all associated functions such as storing and retrieving, locking and transaction management; ie CRUD services. Such a service will manage a single data entity, encapsulating its CRUD operations plus associated business logic.
Business Services	Business Services represent stateless services which directly implement business service capability and provide business value (for example, eligibility for a particular procedure; "am I financial?" etc). Typically coarser-grained than entity services, these encapsulate the data and business logic necessary to deliver a business result. These services may consume entity services or other business services.
Technical Services	Technical services provide support to business functions, rather directly delivering business outcomes. These may be business utility services (ie document storage, mail, and print functions), technical utility services (eg logging, handling technical exceptions), etc. Technical services can be linked to form composite business services, using service assembly or process orchestration.
Business Process services	Process Services are stateful services representing the whole or a fragment of one of the organisation's business processes. These may consume other business process services, business services, entity services, and technical services, and are typically implemented on a BPM platform.
Back end for front end service	A service constructed to provision a particular front-end requirement; typically a composed service using business, entity, technical services for a particular front-end purpose. May be regarded as a specialized form of business service, built for a particular front-end purpose.

The model is an enhancement on the model described earlier, introducing an additional service type – the back end for front end (BEF). The BEF service type is the component amenable to rapid turnover, built to address the needs of the digital age. BEFs are "disposable", with the developer building a new BEF in preference to an updated version where necessary.

These BEFs are built upon a foundation provisioned by the other service types – business, business process, technical. In counterpoint to the BEFs, these "traditional" service types need the more measured development approach; especially in the case of services exposed out of the core systems of a complex business. The BEFs are part of an entire paradigm – typically comprising DevOps and agile development, automated testing, rapid and incremental delivery cycles, cloud provisioning. The traditional service types are part of the familiar "waterfall" approach, with its own tools, techniques and methodology.

For this two world/ two speed approach to be successful, whenever there is a need to a BEF, its source services must be available. We cannot wait six months for a couple of business services that the BEF requires. What needs to happen for the success of this model then is availability; the source services that the BEF requires must exist. To facilitate this two world/ two speed approach, the enterprise must invest in "renovating" their estate of existing services – refactoring them to get rid of technical debt associated with the estate. This activity may be undertaken in advance of need for specific business functionality, as a background foundational activity.

The integration infrastructure for this two speed approach to services is pragmatic: if the organisation already possesses a mature SOA technology infrastructure, continue to use the SOA infrastructure, and use the newer and simpler API protocols (REST/HTTP) for the new digital APIs. If on the other hand the enterprise is relatively new to service management, then employ the API management framework and protocols (given here) to construct the new services of all types and to proxy all services.

VI. Conclusion

Micro-services and SOA are best regarded as allies rather than enemies. Unfortunately the nature or today's enterprises predicate a certain level of complexity, and this complexity is reflected in their systems and represented in the services they expose. Inherent here also is the need to tackle your estate of core systems in a considered, measured fashion (lest you introduce accuracy, reliability, integrity, performance, problems) while at the same time possessing the ability to respond to the needs of the digital world (lest you succumb as a business to competitive pressures evident in that arena). Therefore the most sensible way forward is to embrace both paradigms, and make this possible through seamless integration of the API and SOA styles of service design and delivery, utilising an appropriate service model, one that contains both styles of service and articulates their relationships.

References

[1] Birrell and Nelson, "Implementing Remote Procedure Calls, "ACM Transactions on Computer Systems, Vol. 2, No. 1, February 1984.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.

[2] Gartner, "Hype Cycle for Application Architecture", 2002, 2010, 2015.

[3] Dmitry Namiot, Manfred Sneps-Sneppe. "On Microservices Architecture", International Journal of Open Information Technologies ISSN: 2307-8162 vol. 2, no. 9, 2014, pp.24-27.

[4] Yale Yu, Haydn Silveira and Max Sundaram. "A Microservice based Reference Architecture Model in the context of Enterprise Architecture", IEEE Advanced Information Management, Communicates, Electronic and Automation Control Conference, October 3-5, 2016, Xi' an, China.

[5] Newman, Sam. "Building Microservices", O'Reilly Media, pp.410-420.

[6] Gartner IT glossary: http://www.gartner.com/it-glossary/bimodal/.

Authorized licensed use limited to: Auckland University of Technology. Downloaded on August 21,2021 at 10:25:54 UTC from IEEE Xplore. Restrictions apply.